

Pre-Test: Transactions & Isolation Levels

Nama: **Raihan Muhammad Riswandi**

NIM: **24110500001**

Bagian A – Jawaban Konsep Dasar

1. c) Semua perubahan data sejak BEGIN dibatalkan dan database kembali ke state awal
2. a) Setelah eksekusi statement selesai
3. b) BEGIN membuat transaction block dengan perubahan sementara
4. c) Semua lock yang dipegang transaction dilepaskan
5. c) READ UNCOMMITTED
6. b) REPEATABLE READ mencegah Non-Repeatable Read dengan snapshot consistency
7. b) READ COMMITTED
8. c) Melihat baris baru yang ditambahkan oleh transaksi lain dalam range yang sama

Bagian B – Implementasi Praktis

9. Implementasi transaksi transfer uang:

DO \$\$

DECLARE

 v_source_balance DECIMAL(15,2);

BEGIN

 -- Mulai transaksi

 BEGIN;

 -- Lock baris akun sumber untuk validasi saldo

 SELECT balance INTO v_source_balance

 FROM accounts

 WHERE account_id = 'A001'

 FOR UPDATE;

 -- Validasi saldo cukup

 IF v_source_balance < 500000 THEN

 RAISE EXCEPTION 'Saldo tidak mencukupi untuk transfer';

 END IF;

 -- Debit dari akun sumber

 UPDATE accounts

 SET balance = balance - 500000,

 last_updated = NOW()

 WHERE account_id = 'A001';

 -- Credit ke akun tujuan

 UPDATE accounts

 SET balance = balance + 500000,

 last_updated = NOW()

 WHERE account_id = 'B002';

 -- Logging transaksi berhasil

 INSERT INTO transaction_log (source_account, dest_account, amount, status)

 VALUES ('A001', 'B002', 500000, 'SUCCESS');

```

        -- Commit transaksi
        COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        -- Rollback jika error
        ROLLBACK;
        INSERT INTO transaction_log (source_account, dest_account, amount, status)
        VALUES ('A001', 'B002', 500000, 'FAILED');
        RAISE NOTICE 'Transaksi gagal dan dibatalkan';
END;
$$;

```

Penjelasan: Transaksi dimulai dengan BEGIN, melakukan pengecekan saldo dan penguncian baris (FOR UPDATE), lalu commit jika sukses atau rollback jika gagal.

10. Demonstrasi perbedaan READ COMMITTED vs REPEATABLE READ:

Scenario 1 – READ COMMITTED

```

-- Scenario 1: READ COMMITTED
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN;
SELECT balance FROM accounts WHERE account_id = 'A001'; -- Misal hasil: 1,500,000

```

```

-- Transaction B (session lain)
UPDATE accounts SET balance = 1,000,000 WHERE account_id = 'A001';
COMMIT;

```

```

-- Transaction A (lanjut)
SELECT balance FROM accounts WHERE account_id = 'A001'; -- Hasil: 1,000,000
COMMIT;

```

Ekspektasi hasil: Nilai berubah karena transaksi lain sudah commit (Non-repeatable read terjadi).

Scenario 2 – REPEATABLE READ

```

-- Scenario 2: REPEATABLE READ
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN;
SELECT balance FROM accounts WHERE account_id = 'A001'; -- Misal hasil: 1,500,000

```

```

-- Transaction B (session lain)
UPDATE accounts SET balance = 1,000,000 WHERE account_id = 'A001';
COMMIT;

```

```

-- Transaction A (lanjut)
SELECT balance FROM accounts WHERE account_id = 'A001'; -- Tetap: 1,500,000
COMMIT;

```

Ekspektasi hasil: Nilai tetap sama karena snapshot data tidak berubah selama transaksi berlangsung (mencegah Non-repeatable read).

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Snapshot Konsisten
READ UNCOMMITTED	Ya	Ya	Ya	Tidak
READ COMMITTED	Tidak	Ya	Ya	Tidak
REPEATABLE READ	Tidak	Tidak	Ya	Ya
SERIALIZABLE	Tidak	Tidak	Tidak	Ya